

Weather

Current conditions, 48-hour hourly, 10-day, and animated radar. No ads, no trackers, no analytics, no cookie banner, no account, no API keys.

Two builds, same data and same look:

	Web PWA	Native Android
Where	repo root, served by GitHub Pages	android/
Built with	one HTML + one CSS + one JS file, no build step	Kotlin, Jetpack Compose, osmdroid
Install	Chrome → Add to Home screen	sideload <code>weather.apk</code>
Offline	service worker + localStorage	SharedPreferences cache

Accuracy is ballpark by design — this is a "do I need a jacket / is that cell going to hit Veenker" app, not a life-safety product. Watches and warnings come straight from the National Weather Service and are the one thing here worth trusting.

What's on the screen

Section	What it shows
Alerts	Active NWS watches/warnings for the exact point. Tap one to read the full text. Hidden when there are none.
Current	Temp, condition, feels-like, today's high/low, wind + direction, gust, humidity, UV, sunrise/sunset.
Hourly	Next 48 hours. Horizontal scroll. Blue bar under each hour = chance of precip.
10 Day	Day and date, icon, max chance of precip, and a low→high bar scaled across the days shown. Tap a day to swap the hourly strip to that day; tap again for the rolling 48 h. The <code>5 day / 10 day</code> button in the card header switches the length, and the choice is remembered.
Radar	10 observed NEXRAD frames (last 90 min, 10-minute steps) plus up to 12 HRRR model frames (~3 hr ahead, 15-minute steps), all on the NWS color scale. Opens parked on the newest observed frame — press play to run the loop. Every frame's tiles are fetched as soon as the radar loads, so the first loop draws. Drag the slider to scrub, <code>Expand</code> for fullscreen, pinch to zoom. A <code>+</code> on the timestamp means it's a forecast frame, not an observation. Storm tracks : moving cells (≥ 5 kt, ≥ 40 dBZ) within 300 km get a dot, a line to where they'll be in 60 minutes, ticks at 15 / 30 / 45 min and an arrowhead — red when the radar flags a tornado vortex signature or a $\geq 50\%$ probability of severe hail. If a track passes within 8 km of the place, a line under the timeline says which cell and roughly when (<code>Cell 25 mi W, 46 mph, reaches here $\approx$ 3:02pm</code>). US only — elsewhere the timestamp reads <code>US only</code> .

Location: tap the place name to search any city, or the crosshair/pin to use GPS. The last six places you looked at stay as chips in the search screen. Defaults to Ames, IA on a fresh install.

Install the Android app

On your phone, in Chrome:

1. Open <https://postmaster87.github.io/matts-weather-app/weather.apk>
2. Chrome will warn about the file type — accept the download.

3. Tap the downloaded file. Android will ask to allow installs from Chrome; turn it on, then tap Install.
4. The launcher icon is the sun-behind-cloud, labelled **Weather**.

It is signed with a personal **release** key (4096-bit RSA, CN=Matt), not the Android debug key. Android still calls it an app "from an unknown source" — that is what sideloading looks like — but every later build signed with the same key installs straight over this one and keeps its data. No uninstall step, ever again.

The published `weather.apk` is byte-for-byte the build running on the phone:

```
version 1.2 (versionCode 3)
sha256 20b68ffd4fd152fe6fa71614757cda5c3f9d9812873332ab71d02b6161fd8805
```

The key is two files, neither of them in this repo, and neither recoverable:

```
%USERPROFILE%\androidkeys\matts-weather-release.p12
android/keystore.properties          (holds the keystore password)
```

Back both up somewhere off the build machine. Lose either one and no future build can update an installed copy — the only path left is uninstall and reinstall, which loses the saved places and the cached forecast.

Install the web app instead

On your phone, in Chrome:

1. Open <https://postmaster87.github.io/matts-weather-app/>
2. Menu (:) → **Add to Home screen**

Both keep the last forecast on the device and open offline with an orange **Offline — cached 2:14pm** line at the bottom instead of pretending the numbers are current.

Building the Android app

Everything needed is already on the Windows box: JDK 17, Android SDK 35, build-tools, and Gradle 8.9.

```
cd android && ./gradlew assembleRelease
```

Output lands at `android/app/build/outputs/apk/release/app-release.apk` (~6.8 MB). Copy it to the repo root as `weather.apk` to publish it through Pages.

Two files are needed and neither is committed:

File	Holds
<code>android/local.properties</code>	<code>sdk.dir=<path to the Android SDK></code>
<code>android/keystore.properties</code>	<code>storeFile, storePassword, keyAlias (weather), keyPassword</code>

Without `keystore.properties` the release build comes out **unsigned** rather than quietly falling back to the debug key — Gradle prints a warning saying so, and an unsigned APK refuses to install. That is a much louder failure than one signed with the wrong key, which would only surface later as an update that cannot be applied.

To run it on the emulator:

```
adb install -r android/app/build/outputs/apk/release/app-release.apk
```

Layout

```
android/app/src/main/java/com/matt/weather/
  MainActivity.kt          activity, permission request, theme
  WeatherVm.kt            state, refresh + retry, GPS, search
  data/
    Models.kt             Place, Current, Hour, Day, Forecast, Alert, RadarFrame, StormCell
    WeatherApi.kt         Open-Meteo, NWS, IEM radar + storm cells, geocoding
    StormTracks.kt        storm-track math: filter, de-dupe, extrapolate, arrival
    Net.kt                HttpURLConnection with a real User-Agent
    Store.kt              SharedPreferences: place, recents, cached payload
    Loc.kt                LocationManager fix + platform reverse geocode
    Fmt.kt                wall-clock string formatting
    Wmo.kt                WMO code -> icon + label
  ui/
    WeatherScreen.kt      screen, top bar, fullscreen radar
    Cards.kt              current / hourly / daily / alert cards
    RadarCard.kt          osmdroid map, frame overlays, storm tracks, timeline
    WeatherIcon.kt        the 12 condition icons, drawn on Canvas
    Theme.kt              palette
```

Icons are drawn on a Compose Canvas with the same 24-unit geometry as the web build's SVGs, so both versions look identical rather than merely similar.

Data sources

Source	Used for	Key needed
Open-Meteo	Forecast (hourly + daily)	No
Open-Meteo Geocoding	Place search	No
api.weather.gov	NWS alerts (US only)	No
Iowa Environmental Mesonet (IEM)	Radar tiles: NEXRAD US composite (past) and HRRR simulated reflectivity (future); NEXRAD storm attributes (storm tracks). US only	No
Esri dark canvas	Radar basemap	No
Leaflet 1.9.4 / osmdroid	Map (web / Android)	No

Nothing leaves the phone except the coordinates of the place being looked up. No server, no database, no analytics.

The Android build sends `MattsWeather/1.0 (Android; +<repo URL>)` as its User-Agent because `api.weather.gov` asks callers to identify themselves and throttles generic agents. It deliberately carries **no contact email** — add one there if you ever want NWS to be able to reach you about traffic.

The web build reverse-geocodes the GPS button's label through [BigDataCloud](#); the Android build uses the platform `Geocoder` instead, so it makes no third-party call for that.

Known limits

- **Radar is US-only.** IEM's composite and HRRR cover the lower 48 and a margin around it (lat 21-53, lon -130 to -60). Outside that box the radar timestamp reads `US only` and no radar or storm data is fetched. Map zoom is capped at 10.
 - **Forecast radar frames are HRRR model output, not observation.** They come in 15-minute steps (the observed frames are 10 minutes apart), and the model run behind them is usually 1-3 hours old.
 - **Storm tracks are NEXRAD storm attributes extrapolated in a straight line at constant speed.** Real cells turn, speed up, split and die; the arrival line is a rough heads-up, not a warning.
 - **Alerts are US-only.** `api.weather.gov` returns nothing outside the US; the alert strip just stays hidden.
 - **Radar timestamps are in the phone's timezone**, everything else is in the forecast location's local time. Only matters when looking at another time zone.
 - **Daily icon vs. daily precip %** can look inconsistent (an overcast icon next to 89%). Both come from Open-Meteo as-is: the icon is the day's dominant condition, the % is the single wettest hour. Not smoothed over.
 - **Days 8-10 are the loosest numbers on the screen.** Open-Meteo can return no precip % that far out, so a row can show a rain icon with no percentage.
 - Open-Meteo is a model blend, not a nowcast. Treat the hourly numbers as directionally right, not exact.
 - The Android app is **not on Google Play** and has no update mechanism — a new version means building a new APK and installing it over the old one. Since it is signed with the release key that install is now a plain in-place update; it was an uninstall-and-lose-your-data affair while the debug key was in use.
-

Web build files

<code>index.html</code>	markup
<code>app.css</code>	all styling, dark theme, phone-first
<code>app.js</code>	everything else — data, icons, render, radar, storm tracks
<code>sw.js</code>	service worker, caches the shell only
<code>manifest.webmanifest</code>	PWA manifest
<code>icons/</code>	192 / 512 / maskable-512 PNGs
<code>weather.apk</code>	the published Android build

Local preview:

```
python -m http.server 8765
```

Then open `http://localhost:8765`. A service worker is registered, so after editing `app.js` or `app.css`, hard-reload (or bump `CACHE` in `sw.js`) to be sure you're seeing the new file.